

Model Equations Explained — Step by Step

KM Trans Logistics Workshop Simulator

Term IV Project Course | Faculty Guide: Prof. Debjit Roy | Jayant Gangwar & Pranit Ras

June 2026

Contents

1 Equations Explained — Step by Step	1
1.1 0. The foundational unit: the “working minute”	1
1.2 1. The arrival process — when do trucks show up?	2
1.3 2. Service times — how long does a repair take?	4
1.4 3. Workers and bays — who and where	7
1.5 4. Seasonal absenteeism (festival mode)	8
1.6 5. Output metrics — what we measure	8
1.7 6. Running it many times — replications and confidence	10
1.8 7. Validating against reality (Exhibit 8)	11
1.9 8. One-line recap of the whole chain	11

1 Equations Explained — Step by Step

A teaching walkthrough of **every** equation in the KM Trans workshop simulator. For each one: the equation in LaTeX, what each symbol means, what it computes, and the intuition. Read this top to bottom — each section builds on the previous one.

1.1 0. The foundational unit: the “working minute”

Before any equation, we fix the unit of time. The shift runs 10:00–19:00 with 2 hours of breaks, so a mechanic works about 7 productive hours a day. In minutes:

$$\text{PMD} = 7 \times 60 = 420 \text{ minutes/day}$$

- PMD = **P**roductive **M**inutes per **D**ay.

Check that this is right: the case says an accident takes ~15 days and Exhibit 5 lists its mean service time as 6300 minutes. Dividing:

$$\frac{6300 \text{ min}}{15 \text{ days}} = 420 \text{ min/day} \quad \checkmark$$

So the service times in the case are *already in productive minutes*. We therefore run the entire simulation clock in working minutes, and convert to days whenever we want a human-readable number:

$$\text{days} = \frac{\text{minutes}}{\text{PMD}} = \frac{\text{minutes}}{420}$$

1.2 1. The arrival process — when do trucks show up?

1.2.1 1.1 Arrival rate of one job type

Exhibit 4 counts how many of each job type arrived over the 73-day observation window, split into flatbed and car carriers. The **mean number per day** for job type i is:

$$\lambda_i = \frac{f_i + c_i}{73}$$

- λ_i = average arrivals **per day** of job type i (the Greek letter *lambda*, the standard symbol for a rate).
- f_i = total flatbed-carrier arrivals of type i over 73 days (Exhibit 4).
- c_i = total car-carrier arrivals of type i over 73 days.
- 73 = number of days observed.

What it calculates: how often, on an average day, that specific repair shows up. Example — Wiring had $317 + 134 = 451$ arrivals, so $\lambda_{\text{wiring}} = 451/73 \approx 6.18$ per day.

1.2.2 1.2 Total arrival rate for the whole workshop

Sum over all job types, and allow the user to scale demand up or down:

$$\Lambda = dm \cdot \sum_i \lambda_i$$

- Λ = total arrivals per day across **all** job types (capital lambda).
- dm = demand multiplier (the slider; $dm = 1$ is the case baseline, $dm = 1.5$ is +50% demand).
- $\sum_i \lambda_i$ = add up every job type's daily rate. For this data $\sum_i \lambda_i \approx 30.4$.

What it calculates: the total truck-job inflow per day. At baseline, ~30 jobs/day arrive.

1.2.3 1.3 Converting a daily rate to a per-minute rate

The clock ticks in minutes, so we express the rate per working minute:

$$\lambda_{\min} = \frac{\Lambda}{\text{PMD}} = \frac{\Lambda}{420}$$

- $\lambda_{\min}$ = arrivals per **working minute**.

What it calculates: the chance-density of an arrival in any given minute. At baseline, $\lambda_{\min} \approx 30.4/420 \approx 0.0724$ arrivals per minute ($\approx$ one every 13.8 minutes).

1.2.4 1.4 Time between arrivals — the Exponential distribution

We assume arrivals follow a **Poisson process**: events that are individually rare, independent, and happen at a constant average rate. A fundamental result is that the **gaps between consecutive events of a Poisson process are Exponentially distributed**. The Exponential distribution has this cumulative distribution function (the probability a gap is $\leq t$):

$$F(t) = 1 - e^{-\lambda_{\min} t}$$

- $F(t)$ = probability that the next arrival comes within t minutes.
- e = Euler's number (≈ 2.718); $e^{-\lambda_{\min} t}$ is the probability of *no* arrival in t minutes.

We don't use F directly; we **invert** it to generate random gaps (next equation).

1.2.5 1.5 Sampling a random gap — the inverse-transform method

To turn a uniform random number into an Exponential sample, set $F(T) = U$ and solve for T :

$$U = 1 - e^{-\lambda_{\min} T} \implies T_{\text{gap}} = -\frac{\ln(U)}{\lambda_{\min}}$$

(We use U in place of $1 - U$ since both are Uniform(0,1).)

- T_{gap} = the simulated number of working minutes until the next truck arrives.
- U = a uniform random number in $[0, 1)$ from the seeded generator.
- $\ln$ = natural logarithm.

What it calculates: one random inter-arrival time. The engine schedules the next arrival at *current time* + T_{gap} , then repeats — producing the whole stream of arrivals.

Intuition: small $U \rightarrow$ large gap; U near 1 $\rightarrow$ tiny gap; on average $T_{\text{gap}} = 1/\lambda_{\min}$.

1.2.6 1.6 Which job type is this arrival?

When an arrival occurs, we pick its type with probability proportional to its rate:

$$P(\text{type} = i) = \frac{\lambda_i}{\sum_j \lambda_j}$$

- $P(\text{type} = i)$ = probability this arrival is job type i .
- Numerator = that type's rate; denominator = total rate.

What it calculates: the job mix. Busy job types (Wiring, Pressure Leakage) appear most often.

1.2.7 1.7 Flatbed or car carrier?

Given the type, pick the carrier class using Exhibit 4's split:

$$P(\text{flatbed} \mid \text{type} = i) = \frac{f_i}{f_i + c_i}$$

- The bar “|” reads “given”. This is the probability of flatbed, *given* the type is i .

What it calculates: the carrier class, so we can later apply the “car carriers get priority” rule from the case.

1.2.8 1.8 Sanity-checking the Poisson assumption (over-dispersion)

Exhibit 4 also lists a standard deviation of daily arrivals, s_i . A genuine Poisson count has the special property that its variance equals its mean, so its standard deviation should be $\sqrt{\lambda_i}$. We compare the reported spread to that:

$$D_i = \frac{s_i}{\sqrt{\lambda_i}}$$

- D_i = dispersion ratio for type i .
- s_i = std. deviation of arrivals/day from Exhibit 4.
- $\sqrt{\lambda_i}$ = the std. deviation a Poisson process *would* have.

What it calculates / how to read it: - $D_i \approx 1 \rightarrow$ Poisson fits well. - $D_i > 1 \rightarrow$ arrivals are “burstier” than Poisson (over-dispersed); a Negative-Binomial daily count would be more honest.

This is the “use the exhibit data to justify the distribution” step. The engine uses Poisson by default and we flag the over-dispersed types in the report.

1.3 2. Service times — how long does a repair take?

1.3.1 2.1 Skill changes speed: the skill multiplier

Exhibit 5 times are for an **expert** (skill level 10). The case says a lower-skilled worker is slower in proportion to skill. We capture this with a multiplier:

$$\varphi(s) = \frac{10}{s}$$

- $\varphi(s)$ = “slowness factor” for a worker of skill s (Greek *phi*).
- s = the worker’s skill level on a 0-10 scale.

Mapping Exhibit 2's categories to numbers — **High** = 10, **Medium** = 7, **Low** = 4 — gives:

$$\varphi(10) = 1.0, \quad \varphi(7) \approx 1.43, \quad \varphi(4) = 2.5$$

What it calculates: how much longer than an expert this worker takes. The $\varphi(4) = 2.5$ matches the case statement exactly ("low (4) rated $\rightarrow 2.5\times$ the expert time").

1.3.2 2.2 The governing skill when several workers share a job

Some jobs need more than one worker (possibly different skills). The job is not finished until the **slowest** sub-task is done, so the time is set by the **lowest** skill present:

$$s^* = \min(s_1, s_2, \dots, s_n)$$

- s^* = the governing (worst) skill among the workers assigned to this job.
- $s_1, \dots, s_n$ = the skills of the individual workers seized for this job.

What it calculates: the single skill number that will drive this job's duration.

1.3.3 2.3 The effective mean service time

Combine the base mean with the governing skill's slowness:

$$\mu_{\text{eff}} = m_i \cdot \varphi(s^*) = m_i \cdot \frac{10}{s^*}$$

- μ_{eff} = the mean service time we'll actually use for this job (minutes).
- m_i = Exhibit-5 expert mean time for job type i .

What it calculates: the average duration this job *should* take given who is working on it. An all-low-skill crew makes $\mu_{\text{eff}} = 2.5 m_i$.

1.3.4 2.4 Why Lognormal, and how to set its parameters

Service times must be **positive** and are **mildly right-skewed** (a few jobs run long). The **Lognormal** distribution fits both facts: a variable whose *logarithm* is Normally distributed. A Lognormal built from an underlying Normal with parameters $(\mu_{\log}, \sigma)$ has these two known properties:

$$\mathbb{E}[X] = e^{\mu_{\log} + \sigma^2/2}, \quad \text{CV}^2 = e^{\sigma^2} - 1$$

- $\mathbb{E}[X]$ = the mean of the service time X (we want this to equal μ_{eff}).
- $\mu_{\log}, \sigma$ = the mean and std. deviation of the *underlying Normal* (i.e. of $\ln X$).
- CV = coefficient of variation = (std. deviation)/(mean). We **choose** it from the case: CV = 0.3 for routine jobs, CV = 0.7 for the high-variability ones (accident, denting, engine overhaul).

We have a target mean μ_{eff} and a target CV, and we need $(\mu_{\log}, \sigma)$. Solving the two equations above gives:

$$\sigma^2 = \ln(1 + \text{CV}^2)$$

$$\mu_{\log} = \ln(\mu_{\text{eff}}) - \frac{\sigma^2}{2}$$

- First line: invert $\text{CV}^2 = e^{\sigma^2} - 1$.
- Second line: rearrange $\mathbb{E}[X] = e^{\mu_{\log} + \sigma^2/2}$ for $\mu_{\log}$.

What it calculates: the exact Normal parameters so that the resulting Lognormal has *precisely* the mean and CV we want. This is “moment matching.”

1.3.5 2.5 Drawing one random service time

With $(\mu_{\log}, \sigma)$ in hand:

$$X = \exp(\mu_{\log} + \sigma \cdot Z), \quad Z \sim \mathcal{N}(0, 1)$$

- X = the sampled service time in minutes for this job.
- Z = a standard Normal random number (mean 0, std. dev 1).
- $\exp(\cdot) = e^{(\cdot)}$, which guarantees $X > 0$.

What it calculates: one realistic, random repair duration. Most draws sit near μ_{eff} ; a few land in the long right tail (the occasional job that drags on).

1.3.6 2.6 Generating the standard Normal Z — Box-Muller

Our random generator only produces uniforms, so we convert two uniforms into a Normal using the **Box-Muller transform**:

$$Z = \sqrt{-2 \ln U_1} \cos(2\pi U_2)$$

- Z = a standard Normal sample.
- U_1, U_2 = two independent Uniform(0,1) numbers.
- 2π = full circle in radians (the transform maps the two uniforms onto a 2-D Gaussian and reads off one coordinate). The sin version gives a second independent Z , which we cache.

What it calculates: the bell-curve random number Z that feeds the Lognormal in 2.5.

1.4 3. Workers and bays — who and where

1.4.1 3.1 How many workers a job needs from each department

Exhibit 5 (continued) gives a requirement $r_{i,d}$ for job type i and department d . The number of workers actually seized is:

$$n_d = \begin{cases} \text{round}(r_{i,d}) & \text{if } r_{i,d} \geq 1 \\ 1 & \text{if } 0 < r_{i,d} < 1 \\ 0 & \text{if } r_{i,d} = 0 \end{cases}$$

- n_d = workers taken from department d for this job.
- $r_{i,d}$ = the Exhibit-5 requirement (e.g. Clutch Overhaul needs $r = 2$ Mechanical).
- The middle case handles **accidents**, whose requirements are fractions (see 3.2).

What it calculates: the headcount the job must grab from each trade before it can start.

1.4.2 3.2 Fractional accident labour as a *time-share*

The case explains an accident's decimal (e.g. Balancer = 0.1) as "a worker is needed for only 10% of the job's time." So we seize **one** worker but **hold** them for only that fraction of the job's duration D :

$$\text{hold}_d = \begin{cases} r_{i,d} \cdot D & \text{if } 0 < r_{i,d} < 1 \\ D & \text{otherwise} \end{cases}$$

- hold_d = how long department d 's worker is tied up by this job.
- D = the job's full sampled service time (the X from 2.5).

What it calculates: the worker-release timing. The **bay** is held the full D (an accident blocks the bay the whole time), but a trade only needed 10% of the time is freed after 0.1 D . This is exactly the bay-blocking dynamic the case cares about.

1.4.3 3.3 The start condition (can a waiting job begin?)

A queued job may start only when **both** of these hold at the same instant:

(bay free of the right type) AND ($\text{available}_d \geq n_d$ for every department d)

- available_d = workers currently free in department d .

What it calculates: the gatekeeping rule. If either a bay or any required trade is missing, the job waits; every time a resource frees up, the dispatcher re-checks the queue.

1.5 4. Seasonal absenteeism (festival mode)

The absenteeism slider thins each department's roster, skill bucket by skill bucket:

$$\text{available}_{d,k} = \max\left(0, b_{d,k} - \text{round}(a \cdot b_{d,k})\right)$$

- $\text{available}_{d,k}$ = workers left in department d at skill level k (high/med/low).
- $b_{d,k}$ = the baseline count from Exhibit 2.
- a = absenteeism fraction (slider; e.g. $a = 0.4$ during Diwali).
- $\max(0, \cdot)$ = never go below zero workers.

What it calculates: the reduced workforce during festivals — used to test how badly delays spike when ~40% of mechanics don't show up.

1.6 5. Output metrics — what we measure

We exclude a **warm-up** period (the shop starts empty, which is unrealistic) and score only jobs that arrive afterward. Define the measured window:

$$\text{effMin} = (\text{simDays} - \text{warmupDays}) \cdot \text{PMD}, \quad \text{effDays} = \frac{\text{effMin}}{\text{PMD}}$$

For each completed job k , record arrival a_k , service-start s_k , completion c_k (in minutes).

1.6.1 5.1 Waiting time and time-in-system

$$W_k = s_k - a_k \quad S_k = c_k - a_k$$

- W_k = **waiting** time (queueing delay before service starts).
- S_k = **sojourn** time (total time in the workshop = waiting + service).

The average wait, expressed in days:

$$\overline{W}_{\text{days}} = \frac{1}{N} \sum_{k=1}^N W_k / \text{PMD}$$

- N = number of completed jobs in the measured window.
- $\sum W_k / N$ = the plain average; dividing by 420 turns minutes into days.

1.6.2 5.2 The 90th-percentile wait

$$P_{90} = \text{the value below which 90\% of the } W_k \text{ lie}$$

What it calculates: a “bad-but-not-worst-case” wait. We report it because averages hide tails — P_{90} tells you what the unlucky 1-in-10 truck experiences.

1.6.3 5.3 Throughput

$$\text{throughput} = \frac{N}{\text{effDays}}$$

- **What it calculates:** completed jobs per day. If this equals Λ (the inflow), the system is **stable**; if it's persistently below Λ , work is piling up faster than it clears — the shop is overloaded.

1.6.4 5.4 Utilisation (with window-clipping)

Utilisation = fraction of time a resource is busy. A job occupies a resource over an interval [start, end]; we count only the part inside the measured window so the figure can never exceed 100%:

$$\text{overlap} = \max\left(0, \min(\text{end}, H) - \max(\text{start}, w)\right)$$

- overlap = busy minutes of this occupancy that fall inside the window.
- $H = \text{simDays} \cdot \text{PMD}$ = end of the horizon; $w = \text{warmupDays} \cdot \text{PMD}$ = start of the window.

Then:

$$\text{Bay utilisation} = \frac{\sum(\text{bay overlaps})}{B \cdot \text{effMin}} \quad \text{Dept-}d \text{ utilisation} = \frac{\sum(n \cdot \text{overlap})}{C_d \cdot \text{effMin}}$$

- B = total number of bays; C_d = number of workers in department d (after absenteeism).
- Numerators add up all busy-minutes; denominators are the **maximum possible** busy-minutes (resources $\times$ window length).

What it calculates: how hard bays and each trade are working. A department near 100% is the **bottleneck**; bays idle while a trade is maxed out means *labour*, not space, is the constraint.

1.6.5 5.5 Average backlog (time-weighted queue length)

The queue length $L(t)$ jumps up and down over time. The correct “average number waiting” weights each level by how long it lasted (the area under the $L(t)$ staircase, divided by total time):

$$\bar{L} = \frac{\sum_e L_e \cdot (t_{e+1} - t_e)}{\text{effMin}}$$

- $\bar{L}$ = time-weighted average number of jobs waiting.
- L_e = number waiting during the interval between event e and the next event.
- $(t_{e+1} - t_e)$ = the length of that interval in minutes.

What it calculates: the typical backlog. (This is the same averaging idea behind Little’s Law, $L = \lambda \cdot W$, which links backlog, arrival rate, and waiting time.)

1.6.6 5.6 Delay cost in rupees

The case values a day of truck delay at INR 2,500. We charge that on each job’s waiting time:

$$\text{DelayCost} = \sum_{k=1}^N \frac{W_k}{\text{PMD}} \cdot 2500$$

- W_k/PMD = that job’s wait converted to days.
- $\times 2500$ = the case’s opportunity cost per truck-day.

What it calculates: the headline managerial number — what the current bay/manpower plan costs in rupees of avoidable delay over the measured window. This is what every “what-if” is judged on.

1.7 6. Running it many times — replications and confidence

One simulation run is a single roll of the dice. To get trustworthy numbers we run R independent **replications** (each with a different random seed) and summarise them.

1.7.1 6.1 The mean across runs

$$\bar{x} = \frac{1}{R} \sum_{r=1}^R x_r$$

- $\bar{x}$ = average of a metric (e.g. avg wait) across the runs.
- x_r = the metric’s value in run r ; R = number of replications (slider).

1.7.2 6.2 The sample standard deviation across runs

$$\text{SD} = \sqrt{\frac{1}{R-1} \sum_{r=1}^R (x_r - \bar{x})^2}$$

- SD = how much the metric varies from run to run.
- The $R-1$ (not R) is **Bessel’s correction**, which makes this an unbiased estimate of the true variability from a sample.

1.7.3 6.3 The 95% confidence interval

$$\text{CI}_{95} = 1.96 \cdot \frac{\text{SD}}{\sqrt{R}}$$

- CI_{95} = half-width of the 95% confidence interval.

- 1.96 = the standard-Normal value cutting off the central 95%.
- $SD/\sqrt{R}$ = the **standard error** of the mean (the mean's own uncertainty; it shrinks as you do more runs).

What it calculates: the band around $\bar{x}$ within which we're ~95% confident the true mean lies. Reported as $\bar{x} \pm CI_{95}$. More replications → tighter band → more credible result. The KPI cards show exactly this.

Reproducibility note: each replication r uses seed $seed + r \cdot 7919$ (7919 is a prime, chosen so the random streams don't overlap). Because the generator is seeded, re-running with the same seed gives identical results — essential for a report and for fair A-vs-B comparisons.

1.8 7. Validating against reality (Exhibit 8)

Exhibit 8 gives the real share of 340 trucks spending < 1 , > 1 , > 2 , > 5 , > 20 days in the workshop. We compute the same buckets from the simulation and compare as **percentages** so the two are on the same scale:

$$\text{bucket \%} = \frac{(\text{count of jobs in that bucket})}{(\text{total jobs})} \times 100$$

We classify each job by its sojourn in days, S_k/PMD , into the same thresholds.

What it calculates / why it matters: if the simulated percentages reproduce the case's ($\approx 91\%$ finish under 1 day), the model is credible. These buckets are **not** tuned — they're an independent reality check on all the assumptions above. Large gaps tell us which assumption (the CV, the skill mapping, the bay policy) to revisit.

1.9 8. One-line recap of the whole chain

$\lambda_i \Rightarrow \Lambda \Rightarrow T_{\text{gap}}$	(when trucks arrive)
$\varphi(s^*), \mu_{\text{eff}}, X$	(how long the repair takes)
$n_d, \text{hold}_d, \text{start rule}$	(grab a bay + the right workers)
$W_k, \bar{L}, \text{utilisation}, \text{DelayCost}$	(measure performance)
$\bar{x} \pm CI_{95}$	(report it with confidence)

Every symbol in that chain is defined above, and every number traces back to a KM Trans case exhibit.